SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CODING CHALLENGE

Course Code:	CSA0613	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO4 – Precision (BL4,BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge(BL4,BL5)
Weightage:	10% of CIA	**Total Marks:**	2 * 50= 100
CO4 Statement:	*Solve optimization problems using dynamic programming and greedy strategies. (BL4, BL5)*		
Student Name:	P Hemanth	**Reg. No.:**	192472151
Section / Batch:	A/2024	**Date:**	

Code of Conduct

I _P Hemanth(192472151)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P Hemanth _____

Q27. Longest Palindromic Subsequence
Given a string, find the length of the longest subsequence that is also a palindrome. The subsequence need not be contiguous but must maintain the original order. The objective is to determine the maximum possible palindrome length. Use Dynamic Programming to compute the result efficiently.
Input Format:
s = input string
Sample Input:
s = BBABCBCAB

Longest Palindromic Subsequence Using Dynamic Programming

1. Problem Statement

Given a string, find the length of the longest subsequence that is also a palindrome. A subsequence is formed by deleting zero or more characters from the original string without changing the order of the remaining characters.

The subsequence does not need to be contiguous. The objective is to determine the maximum possible length of a palindromic subsequence using Dynamic Programming.

Sample Input:

BBABCBCAB

Expected Output:

Length = 7

One possible longest palindromic subsequence is BACBCAB.

2. Objective

The main objective of this problem is to design and implement an efficient Dynamic Programming algorithm to find the Longest Palindromic Subsequence (LPS) of a given string.

The implementation aims to:

- Identify the longest subsequence that forms a palindrome.
- Maintain the original order of characters.

3. Algorithm Used

The problem is solved using **Dynamic Programming**.

A two-dimensional array dp[i][j] is used, where dp[i][j] represents the length of the longest palindromic subsequence between index i and index j.

The algorithm works as follows:

1. Read the input string.
2. Find the length of the string.
3. Create a two-dimensional DP table of size n × n.
4. Initialize dp[i][i] = 1 because every individual character is a palindrome of length 1.
5. Consider substrings of increasing length.
6. If the characters at both ends are equal, add 2 to the result obtained from the inner substring.

7. If the characters are different, select the maximum value by excluding either the left or right character.
8. The value dp[0][n-1] gives the length of the Longest Palindromic Subsequence.
9. The program also reconstructs and displays one possible longest palindromic subsequence.

4. Dynamic Programming Recurrence

If the characters at positions i and j are equal:

dp[i][j] = dp[i+1][j-1] + 2

If the characters are different:

dp[i][j] = max(dp[i+1][j], dp[i][j-1])

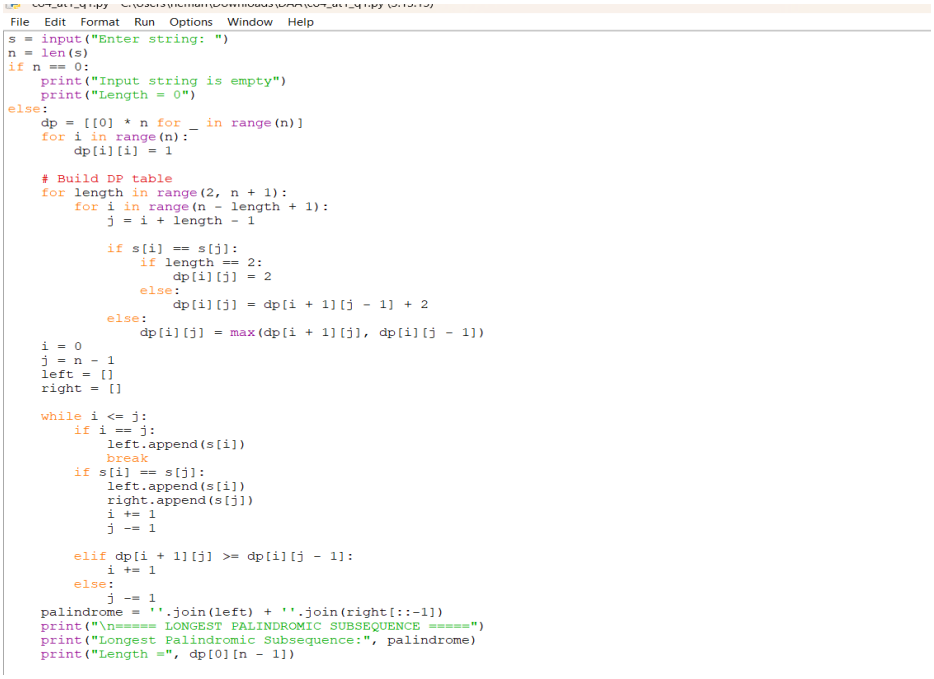
For a single character:

dp[i][i] = 1

Therefore, the final answer is stored in:

dp[0][n-1]

5 Source Code



File Edit Format Run Options Window Help

```
s = input("Enter string: ")
n = len(s)
if n == 0:
    print("Input string is empty")
    print("Length = 0")
else:
    dp = [[0] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = 1

    # Build DP table
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                if length == 2:
                    dp[i][j] = 2
                else:
                    dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    i = 0
    j = n - 1
    left = []
    right = []

    while i <= j:
        if i == j:
            left.append(s[i])
            break
        if s[i] == s[j]:
            left.append(s[i])
            right.append(s[j])
            i += 1
            j -= 1

        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    palindrome = ''.join(left) + ''.join(right[::-1])
    print("\n===== LONGEST PALINDROMIC SUBSEQUENCE =====")
    print("Longest Palindromic Subsequence:", palindrome)
    print("Length =", dp[0][n - 1])
```

6 Output

7 Time and Space Complexity

Time Complexity

The DP table contains approximately n^2 states, and each state requires constant time to calculate.

Therefore:

Time Complexity = $O(n^2)$

Space Complexity

The program uses a two-dimensional DP table of size n × n.

Therefore:

Space Complexity = $O(n^2)$

This is an efficient approach compared with generating all possible subsequences, which would require exponential time.

Conclusion

The Dynamic Programming technique provides an efficient solution for finding the Longest Palindromic Subsequence of a string. The algorithm divides the problem into smaller overlapping subproblems and stores their results in a DP table.

The implementation not only calculates the maximum palindrome length but also reconstructs one possible longest palindromic subsequence. The solution is correct, efficient, and suitable for competitive programming constraints.